

Potext Developer Guide 0.3.0

Chris Ahlstrom
(ahlstromcj@gmail.com)

May 23, 2026



Pseudo-Greek Transliteration

Contents

1	Introduction	3
1.1	Additions to Tinygettext	3
1.2	Unused GNU Gettext Features	4
1.3	Code Changes	4
1.4	Future Work	4
1.5	Naming Conventions	5
1.6	Home Potext Configuration	6
2	Potext Usage in Applications	6
2.1	Main Module Using Potext	7
2.2	Marking a Module for Translation	8
2.3	Creating the .po Files	9
2.4	Installing the .po Files	10
3	Potext Architecture	10
3.1	Logstream Class	12
3.2	Dictionary Manager Class	13
3.3	Pomoparserbase Class	14
3.4	Poparser Class	15
3.5	Moparser Class	16
3.6	Extractor Class	17
3.7	Dictionary Class	17
3.8	Pluralforms Class	19
3.9	Language Class	19
3.10	NLS Bindings Class	20
3.11	Gettext Module	21
4	GNU Gettext and Its Potext Replacements	21
4.1	GNU Gettext Header File	22
4.2	Gettext Module	22
5	Potext Tests	24
5.1	Hello World	25
5.2	Hello Potext	26
5.3	PO Parser Test	26
5.4	MO Parser Test	27
5.5	Potext Test	27
5.5.1	Potext Test 'Translate'	27
5.5.1.1	Translate Basic: Po File and Sample Message	27
5.5.1.2	Translate: Po File, Context, and Sample Message	28
5.5.1.3	Translate: Po File with Singular and Plurals	29
5.5.1.4	Translate: Po File with Context, Singular, and Plurals	29
5.5.2	Potext Test 'Directory'	30
5.5.3	Potext Test 'Language'	30
5.5.4	Potext Test 'Language Directory'	30
5.5.5	Potext Test 'List Message Strings'	30

6	Summary	30
7	References	31

List of Figures

1	Potext "Big Picture" Architecture	11
2	Log Stream (po::logstream)	12
3	Dictionary Manager (po::dictionarymgr)	13
4	PO/MO Parser Base (po::pomoparserbase)	14
5	PO Parser (po::poparser)	15
6	MO Parser (po::moparser)	16
7	Binary Data Extractor (po::extractor)	17
8	Dictionary (po::dictionary)	18
9	Language (po::language)	20
10	NLS Bindings (po::nlsbindings)	21
11	Gettext Module (namespace po)	23

1 Introduction

The *potext* libraries adopt, refactor, and greatly extend the *tinygettext* library ([6]). The purpose of that library is to provide a lightweight mechanism to do translations using the *Portable Object* translation files, *.po, directly. Also incorporated are some elements of the *simple-gettext* library ([5]) to support handling *Machine Object* files, *.mo.

The purpose of the *Potext* library is to provide C++ functions that mirror the many functions of *gettext*, including `textdomain()`, `bindtextdomain()`, `bind_textdomain_codeset()`, `gettext()`, `dgettext()`, `ngettext()`, and others, for translations made via *GNU PO* files and *GNU MO* files. In addition, some of the details of *tinygettext* are wrapped so that, other than marking the text to be translated, the translation setup is done by calling a single function in `main()`.

Our main goal is to make it easy and lightweight to internationalize an application while sticking with *GNU Gettext* conventions. The *GNU Gettext* manual ([2]) is an important resource used in writing *Potext*. Also important is the source code at savannah.gnu.org ([1]).

Note that *Potext* requires the usage of C++17 and above. It support builds using the *GNU* and *Clang* compilers. It support cross-builds using the *MingW* toolset. It also requires the use of *Meson* 1.4 and above. See the `INSTALL` file. *Windows* support (apart from *Mingw* and *Cygwin*) will be provided, but it is not ready yet.

1.1 Additions to Tinygettext

- Re-implementations of *gettext/libintl* functions as a module (collection of functions) in the `po` namespace.
- Integration of the functions above with the dictionary-manager class.
- Support for selecting a domain during the run.

- Support for selecting a `.po/.mo` file directly and getting domain information from it.
- A new class, `nlbindings`, to supplement the `language` class.
- Full documentation of architecture and usage.
- *Meson* `.wrap` files to use *Potext* as a subproject. A sample application project is stored in the tar-file noted below.
- With version 0.2, *Potext* can also read *GNU* `.mo` files for more complete compatibility with *GNU Gettext*.
- Additional test programs, test `.po` files, test `.mo` files, and upgrades to existing tests.

With these additions, *Potext* should be relatively straightforward to use in a new C++ application. The tar-ball `extras/code/mini-potext-test.tar.xz` contains a simple application using *Potext* as a sub-project stored in *GitHub*. Unpack the tar-ball into its own directory and build it by running its `work.sh` script.

1.2 Unused GNU Gettext Features

For ease of use, some features of *GNU Gettext* are not implemented:

- Thread locks in `gettext()`-like functions; locking can be added if testing reveals it necessary for the use-cases that *Potext* supports.
- Determining if the binary is running SUID root.
- All kinds of C macros to choose build options.
- Detecting changes to localization and character conversion environment variables.

Currently *Potext* is meant to be set up once at the start of the run of an application. It assumes that once the setup is made, no localization changes will be made. Keeps it simple.

1.3 Code Changes

- Changed the coding style and naming conventions for (perhaps) easier reading.
- Use of initializer lists, rather than brute-force assignment statements, for initializing various containers.
- Use of the `auto` keyword in declarations and `for`-loops.
- Many additional `using` directives, most hidden in the `po` namespace or in a class declaration.
- A few additional helper classes have been added to provide new features.

These changes are meant to make the code easier to read, understand, and modify.

1.4 Future Work

- Hammer on this code in *Windows*.
- Work out the installation process; including `.po` and `.mo` file installation and copying (if desired) to the user's configuration directory. Add support to assist a *Potext*-using project with the installation of the `.po` files.

- Handle capitals, punctuation, etc. without additional `.po/.mo` entries.
- Get the handling of categories (e.g. `LC_TIME`) to work. However, note that the category is almost always `LC_MESSAGES`, so this is a low priority.
- Allow for the user to override the character set via the `OUTPUT_CHARSET` environment variable.
- Analyze the translation file to determine if the actual character-set matches the specified character-set.
- Support the `"//IGNORE"` and `"//TRANSLIT"` flags for character-set conversions.
- Handle the `"C"` locale as discussed below.

Ignore `LANGUAGE` and its system-dependent analog if the locale is set to `"C"` because:

1. `"C"` locale uses the ASCII encoding; most international messages use non-ASCII characters, which get displayed as question marks or as invalid 8-bit characters.
2. The precise output of some programs in the `"C"` locale is specified by POSIX and should not depend on environment variables like `LANGUAGE`. Such programs can use `gettext()`.

Also ignore `LANGUAGE` and its system-dependent analog if the locale is `C.UTF-8` or `C.<encoding>`; that's the by-design behaviour for `glibc`, see <https://sourceware.org/glibc/wiki/Proposals/C.UTF-8>. Also look in `/usr/lib/locale/C.utf8`.

1.5 Naming Conventions

Potext uses some conventions for naming things in this document.

- **\$prefix**. The base location for installation of the application and its ancillary data files on *UNIX/Linux/BSD*:
 - `/usr/`
 - `/usr/local/`
- **\$winprefix**. The base location for installation of the application and its ancillary data files on *Windows*.
 - `C:/Program Files/`
 - `C:/Program Files (x86)/`
- **\$podir**. The installed location of the `*.po` files. The directory **share** (*Linux*) or **data** (*Windows*), the package-name of the application (**PACKAGE**), and **po** are concatenated, and again the conventions differ between operating systems.
 - `/usr/share/PACKAGE/po/`
 - `/usr/local/share/PACKAGE/po/`
 - `C:/Program Files/PACKAGE/data/po/`
 - `C:/Program Files (x86)/PACKAGE/data/po/`
- **\$home**. The alternate installed location of the `*.po` files. Not to be confused with `$HOME`, this is the standard location for configuration files. On a UNIX-style system, it would be `$HOME/.config/appname`. The files would be put into a **po** subdirectory here.
- **\$localedir**. The installed location of the `*.mo` files. The directory **share** (*Linux*) or **data** (*Windows*), the package-name of the application (**PACKAGE**), and **locale** are concatenated. The conventions for *Linux* versus *Windows* differ as a matter of historical interest:

- /usr/share/PACKAGE/locale/
- /usr/local/share/PACKAGE/locale/
- C:/Program Files/PACKAGE/data/locale/
- C:/Program Files (x86)/PACKAGE/data/locale/

At present, *Potext* does not support directories of *.mo* files. It might, in the future.

- `$modir` and `LC_MESSAGES`. A more common convention for **.mo* files on *UNIX* is to put them in
 - /usr/share/locale/<language>/LC_MESSAGES/PACKAGE.mo
 - /usr/local/share/locale/<language>/LC_MESSAGES/PACKAGE.mo.

Currently, the *Potext* library uses only the **.po* files. In the future it might also handle **.mo* files. Also note that various applications differ in the exact location of their translation files.

1.6 Home Potext Configuration

The *Potext* library also supports installing the **.po* and **.mo* translation files in the user's configuration area. The conventions we use are:

- `$home`. The location where `PACKAGE` installs, creates, or copies its configuration files. Do not confuse it with `$HOME`, although `$home` is in `$HOME/.config/PACKAGE`. The **.po* files are stored in `$HOME/.config/PACKAGE/po`.
- `$winhome`. This location is different for *Windows*: `C:/Users/user/AppData/Local/PACKAGE`. Again, the **.po* files are in a subdirectory called `po`, and the **.mo* files are in a subdirectory called `mo`. or `locale`.

We are still working out details of how best to manage translations for an application.

Also, for reference, we mention some of the files used by *GNU Gettext*.

- `PACKAGE.pot`, created by `xgettext`.
- `LANG.po`, created by `msgmerge`, copying `PACKAGE.pot`, or by editing.
- `LANG.gmo`, created by `msgfmt`.
- For installed packages, see `$prefix/locale/LANG/PACKAGE.mo`.
- Or see `$prefix/locale-langpack.LC_category` (e.g. `LC_NUMERIC`).
- `LANG/PACKAGE.po`, reverse engineered from `PACKAGE.mo` by `msgunfmt`.

Also refer to the *Python* packages *polib* ([4]) and *poedit* ([3]).

2 Potext Usage in Applications

This section briefly covers the usage of *Potext* in an application. A real sample is included in `library/tests/hellopotext.cpp` (see section 5.2 "Hello Potext" on page 26). A small sample application showing the usage of *Potext* as a *Meson subproject* in a completely separate application project is contained in:

```
extras/code/mini-potext-test.tar.xz
```

Unpack this file in its own directory and check it out.

2.1 Main Module Using Potext

The first thing is to add the following header file to the module defining the `main()` function.

```
#include "po/potext.hpp"    // includes "po/gettext.hpp"
```

For clarity, `potext.hpp` is better, but it does include an extra header file.

If *Potext* support is optional for the project, then do something like this; `PROJECT_USE_POTEXT` is a macro optionally defined when configuring the project build.

```
#if defined PROJECT_USE_POTEXT
#include "po/potext.hpp"    // includes "po/gettext.hpp"
else
#define _(str)              (str)
#define N_(str)             str
#endif
```

An application using "gettext" internationalization generally needs to call `setlocale()`, `textdomain()`, and `bindtextdomain()`. The following function declared in the `gettext.hpp` header file wraps up these functions in one call.

```
std::string init_app_locale
(
    const std::string & arg0,
    const std::string & pkgname,
    const std::string & domainname,
    const std::string & dirname,
    const std::wstring & wdirname = L"",
    int category = (-1)
)
```

arg0. The path-name by which the program was called. This information can determine more precisely where installed `.po` files might be stored.

pkgname. The name of the PACKAGE, which can be the short name for the program, such as "helloworld".

domainname. The base name of a message catalog, such as "en_US". It must consist of characters legal in filenames. An application might want to use its package name, such as "helloworld", for the domain name. If empty, then the environment variable `TEXTDOMAIN` is used. If that's empty, then `LC_ALL` is used. If that's empty, then `LC_MESSAGE` is used. Lastly, if that's empty, then `LANG` is used.

dirname. Provides the name of the `LOCALEDIR`. The standard search directory is `/user/share/locale`. If empty, then the environment variable `TEXTDOMAINDIR` is used. If the name is "user", then the `.po` files are searched for in `/home/user/.config/package/` or `C:/Users/user/AppData/Local`, instead of some place in the system.

wdirname. The wide-string version of the locale directory name, most useful in *Windows*. The use of the wide-string parameter is optional, and not well tested yet.

category. The area that is covered, such as `LC_ALL`, `LC_MONETARY`, and `LC_NUMERIC`. The default value selects `LC_ALL`.

The following calls are made for the setup:

1. `std::setlocale()` sets the application's current category and locale. The category is `LC_ALL` by default, and the locale is empty, so that the locale parts are modified according to environment variables.
2. `po::set_locale_info()` sets up the domain name and the locale directory name. If empty, the environment variables discussed above are used. In addition, it is determined if the locale directory is a system directory, the user's configuration directory, or some arbitrary directory containing `.po` files.
3. `po::init_lib_locale()` first asks the dictionary-manager (see section 3 "[Potext Architecture](#)" on page 10) to add all of the dictionaries (`po` files) in that directory to the list of selectable dictionaries, making one of them the default dictionary. Then the reimplementation of the `bindtextdomain()` is called to create a new domain-to-directory binding, and it is inserted into a container. This container supports looking up the locale directory associated with a domain.
4. `po::textdomain()` This function sets the current domain for the dictionary manager to use.

2.2 Marking a Module for Translation

The basic usage to *Potext* is essentially identical to that of *GNU Gettext*, except that (currently) only `.po` files are used directly.

Add the following header file.

```
#include "po/potext.hpp"      // includes "po/gettext.hpp"
```

Mark each translatable string as usual, using the `_()` macro:

```
std::string errmsg = _("Unknown system error");
```

That macro hides a call to `po::gettext()`. Additional "get-text" functions are listed in the table in the following section: section 4.2 "[Gettext Module](#)" on page 22.

2.3 Creating the .po Files

After marking the files that will provide translations, they must be processed to extract the marked strings for translation. For example:

```
$ xgettext test_helpers.cpp --keyword="_" --output="es.po"
```

The result is an untranslated template, `es.po`.

```
# SOME DESCRIPTIVE TITLE.
# Copyright (C) YEAR THE PACKAGE'S COPYRIGHT HOLDER
# This file is distributed under the same license as the PACKAGE package.
# FIRST AUTHOR <EMAIL@ADDRESS>, YEAR.
#
#, fuzzy
msgid ""
msgstr ""
"Project-Id-Version: PACKAGE VERSION\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2024-03-20 06:53-0400\n"
"PO-Revision-Date: YEAR-MO-DA HO:MI+ZONE\n"
"Last-Translator: FULL NAME <EMAIL@ADDRESS>\n"
"Language-Team: LANGUAGE <LL@li.org>\n"
"Language: \n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=CHARSET\n"
"Content-Transfer-Encoding: 8bit\n"

#: test_helpers.cpp:79
msgid "output"
msgstr ""
. . .
```

The next step is to edit this file appropriately, as in the following snippet:

```
# Mensajes en español para test_helpers.
# Copyright (C) 2024 Potext Software Foundation Inc.
# This file is distributed under the same license as the test_helpers package.
# Chris Ahlstrom <ahlstromcj@gmail.com>, 2024.
msgid ""
msgstr ""
"Project-Id-Version: Potext test_helpers 0.1.0\n"
"Report-Msgid-Bugs-To: ahlstromcj@gmail.com\n"
"POT-Creation-Date: 2023-09-18 22:55+0200\n"
"PO-Revision-Date: 2024-03-20 17:08+0200\n"
"Last-Translator: Google Translate <translate.google.com>\n"
"Language-Team: Spanish <es@tp.org.es>\n"
"Language: es\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8-bit\n"
"X-Bugs: Report translation errors to the Language-Team address.\n"
"Plural-Forms: nplurals=2; plural=(n != 1);\n"
```

```
#: test_helpers.cpp:79
msgid "output"
msgstr "producción"
. . .
```

In the project tree, create a `po` directory and move the `.po` file to it.

Note that the *GNU Gettext* manual ([2]) (in chapter 6) describes many more facets (heh heh) to the creation and manipulation of `.po` files.

2.4 Installing the .po Files

The installation process for the project should include installing the `.po` files. Where to install them in the system, if desired? It does not quite make sense to store them in a place like

```
/usr/local/share/locale/LL/LC\_MESSAGES}
```

because that contains `.mo` files (and some *Qt* `.qm` files!).

We would suggest something like `/usr/local/share/po/PACKAGE`. We should provide support in *Potext* for that. Once *Potext* supports parsing `.mo` files, the usual processes and location can be used. Future stuff.

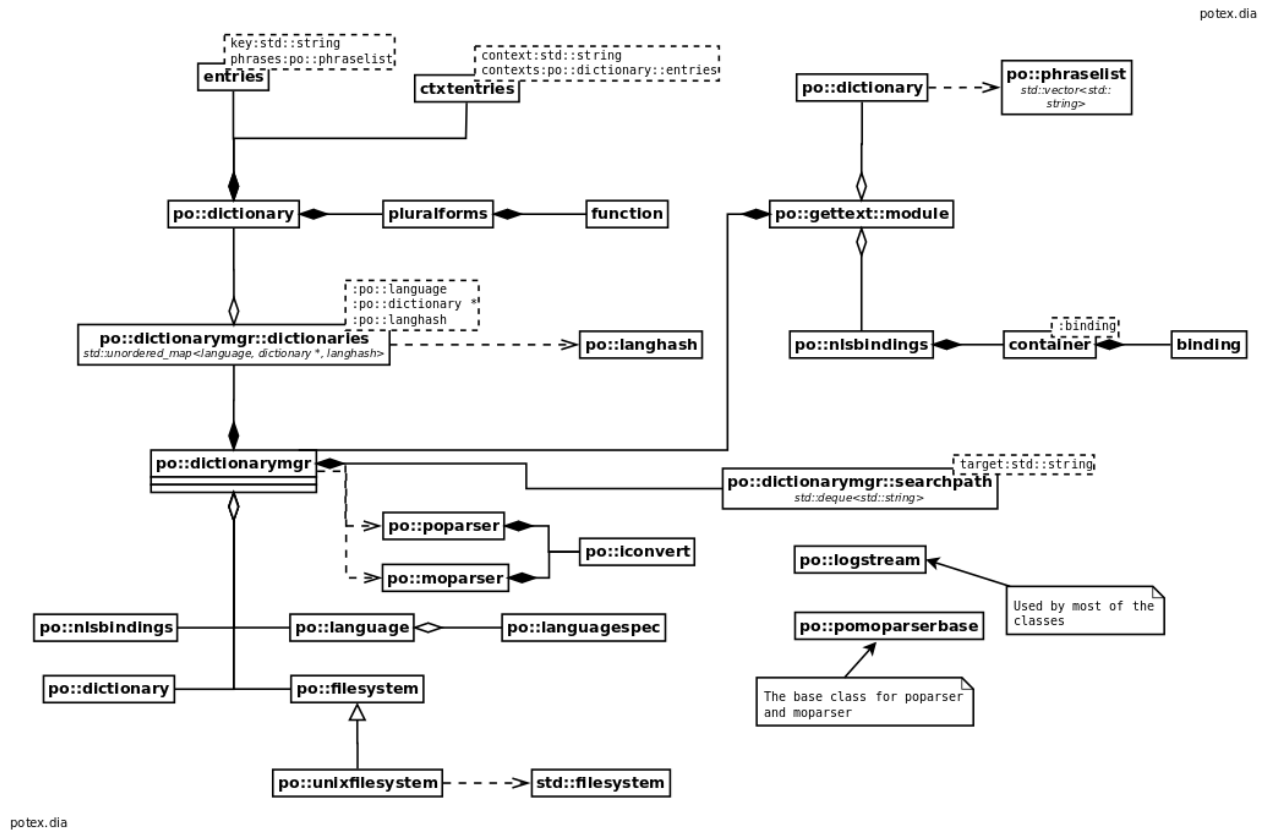
The project, once installed, can also, if desired, copy the relevant language file to the user's configuration directory, `/home/user/.config/package/` or `C:/Users/user/AppData/Local` at the first run, and use it there.

3 Potext Architecture

This section provides a walk-through of the architecture of the *Potext* library. Much of the architecture is similar to *Tinygettext* ([6]), but there are some important changes and additions. Some notes about the classes and documentation:

- All classes and free functions are wrapped in the `po` namespace.
- The macro `_()` that normally wraps *GNU* function `gettext()` here wraps the *Potext* function `po::gettext()`.
- The related "gettext" functions are redefined in terms of *Potext* functions.
- In the diagrams, for the function parameters, we use `"std::string"`, rather than `"const std::string &"` for brevity in the diagrams.
- Not every attribute or function is described. Some groups of items include `_xxxx_` to represent a number of similar functions.
- The copy constructors, principal assignment operators, and destructors are not described. See the header files to see if they are **default**, **delete**, or **defined**.

First, the big picture.



The most important part of *Potext* is the `dictionary` class. It is filled by the `po::poparser::parse()` function, which takes a `.po` file and fills a dictionary object with a set of message strings and their translations. It also includes plural forms from the `.po` file to translate plural messages using "plural" functions. Each dictionary includes the message entries and context entries. For a description of the `.po` file, see the *GNU Gettext* manual ([2]).

The `dictionarymgr` class handles one or more dictionaries. It has been augmented to hold a new `nlsbindings` class to provide support for `bindtextdomain` and `textdomain`, which are *not* provided by *Tinygettext*.

The `dictionarymgr`'s additional member functions are used to implement the free functions in the `gettext` module, such as `gettext()` and `dgettext()`, etc. The `gettext` module is an addition to *Potext* to make it easy to switch from *Gettext* to *Potext*.

The `language` class supports the various parts of a domain name: language, country, modifier, long name, and the long name localized.

Parsing `.po` files is facilitated by the various file-system classes. Each `.po` file results in the creation of a dictionary.

The `poparser` class supports reading and parsing a `.po` file to create a `dictionary`. It inherits some common code from the `pomoparserbase` class.

The `moparser` class supports reading and parsing a `.mo` file to create a `dictionary`. It also inherits

some common code from the `pomoparserbase` class.

The `iconvert` class supports converting the translations to another codeset (besides UTF-8) when creating the dictionaries.

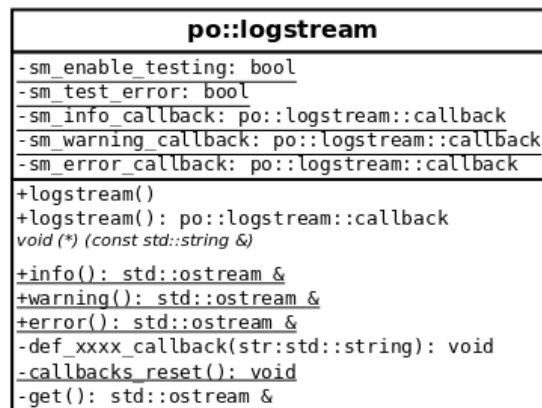
The `logstream` class supports internal error logging, but can also be used by an application.

These classes are discussed in more detail in the following sections.

3.1 Logstream Class

The `po::logstream` class is a reimplementaion of the `tinygettext::Log` class.

logstream.dia



logstream.dia

Figure 2: Log Stream (`po::logstream`)

The `po::logstream` class provides `std::ostream` objects for emitting errors, warnings, and information messages. It also provides the ability to set a callback function to change how the messages are emitted. It is used internally for writing status to the console. It can also be used by an application, but ...

... An interesting issue that we have not yet figured out is illustrated by the test application `hellopotext`. When run, all of the messages written to `std::cout` appear first, including the final message "SUCCESS". Then all of the messages logged during setup and translation in the *Potext* library appear when `hellopotext` is *exiting*.

3.2 Dictionary Manager Class

The `po::dictionarymgr` class is a reimplementation of the `tinygettext::dictionary_manager` class. It contains an `std::unordered_map` of shared pointers to dictionary objects, keyed by language objects which are searched using `operator ()` hash function in a `po::langhash` structure.

Currently, *Potext* does not do anything special with the `searchpath` deque.

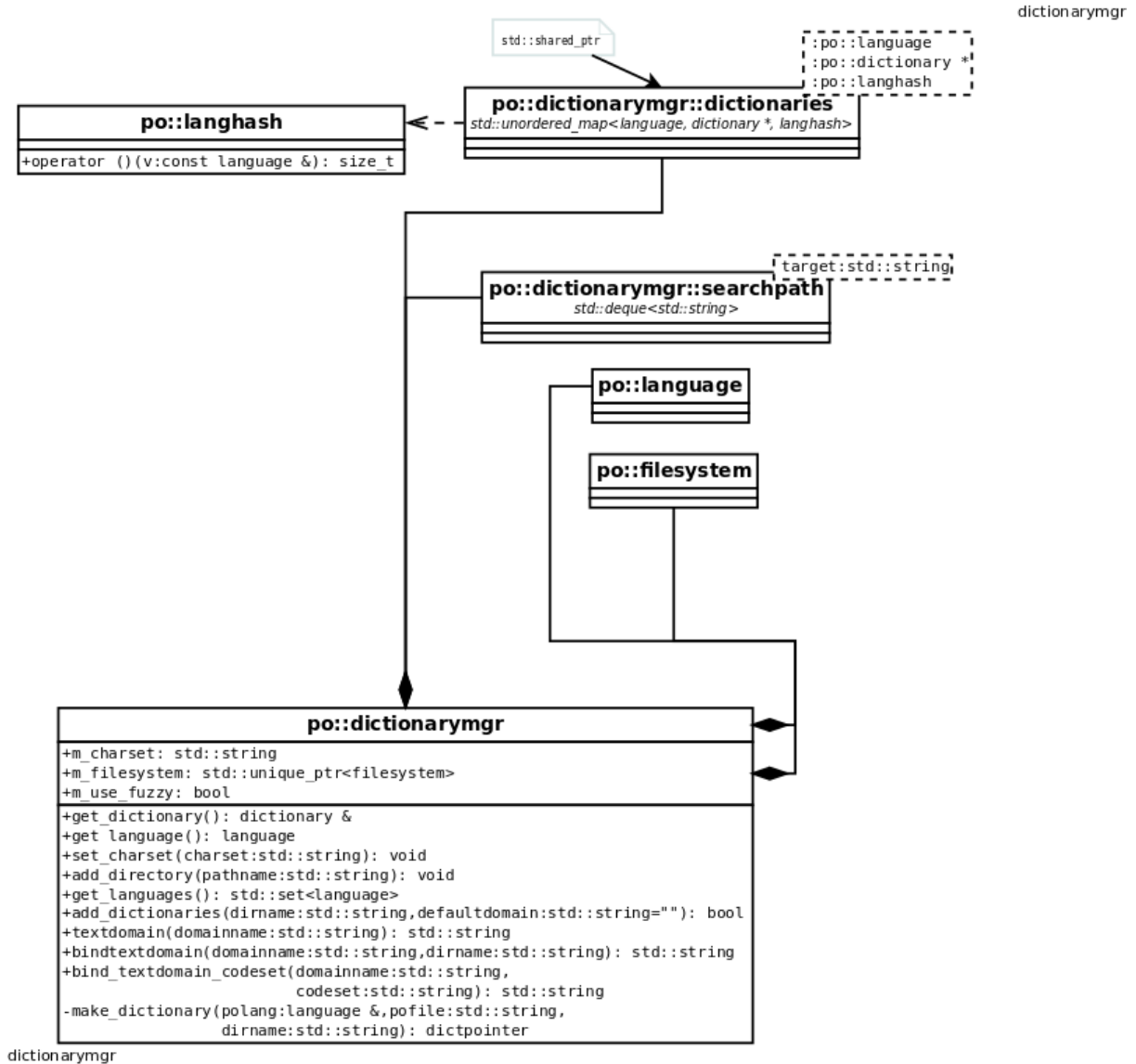


Figure 3: Dictionary Manager (`po::dictionarymgr`)

In *Tinygettext*, the `dictionary_manager` class coordinated multiple locale directories and the selection of a particular dictionary for a translation.

Potext's `dictionarymgr` currently handles only one directory, but it adds support for domain-binding and for actually using translation functions that accept a domain parameter. The new functions are

described next.

- **get_bindings()**. This function returns an **nlsbindings** class reference that contains a list of domain names with the names of the directory and the character-set for each domain. The **nlsbindings** class provides the set-binding functions needed by the following new functions.
- **add_dictionaries()**. This function scans a directory for **.po** files and creates a **dictionary** for each one.
- **make_dictionary()**. This helper function opens a file using the **std::filesystem** class function **open_file**. It then creates an **std::shared_ptr** for a new **dictionary** and calls the static function **po::poparser::parse_po_file()**. Then it calls the member function **po::nlsbindings::set_binding_values()** to create a corresponding binding object.
- **textdomain()**. This function sets the current domain to the given domain name. It is used in the **gettext** module to implement the **textdomain()** function.
- **bindtextdomain()**. This function associates a domain name with a locale directory in which to find the **.po** file. It is used in the **gettext** module to implement the **bindtextdomain()** function.
- **bind_textdomain_codeset()**. This function associates a domain name with a character-set to use in converting messages. It is used in the **gettext** module to implement the **bind_textdomain_codeset()** function.
- **get_dictionary()**. This function returns a reference to the current dictionary object. It is used in the **gettext** module to access the main dictionary to use it in the various **gettext**-like functions.

3.3 Pomoparserbase Class

As of version 0.2, some common functionality has been moved into this base class.

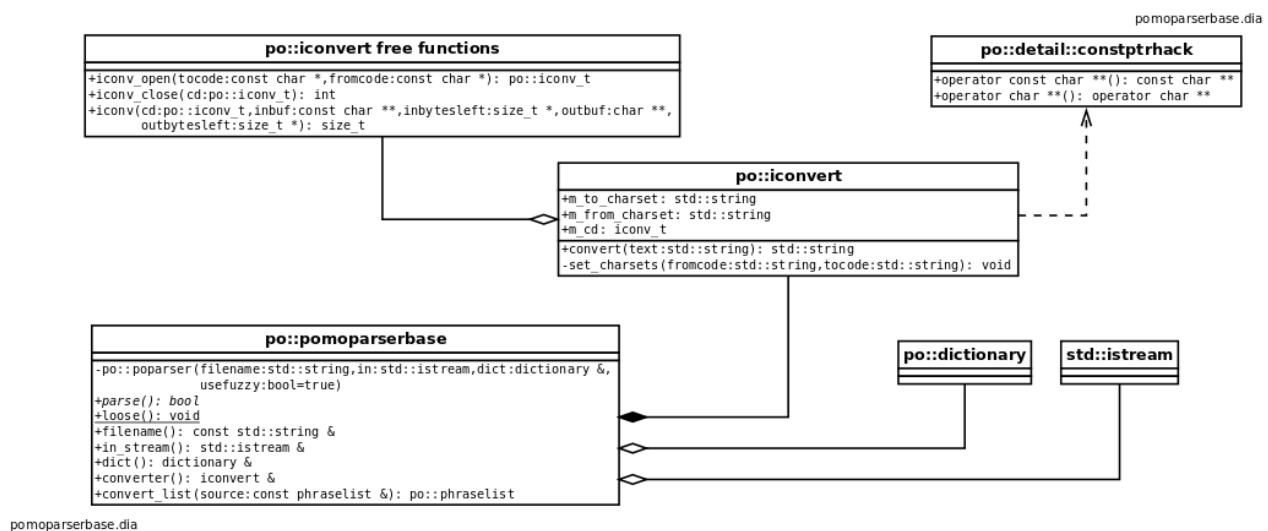


Figure 4: PO/MO Parser Base (po::pomoparserbase)

For version 0.2, we have moved the file-reading support, dictionary class, and **iconvert** class into a base class common to the **moparser** and **poparser** classes.

Other than that commonality, the two parser class are somewhat different in how they parse, since the `.po` file is text, and the `.mo` file is binary.

3.4 Poparser Class

The `po::poparser` class is a serious reimplementaion of the `tinygettext::POParser` class.

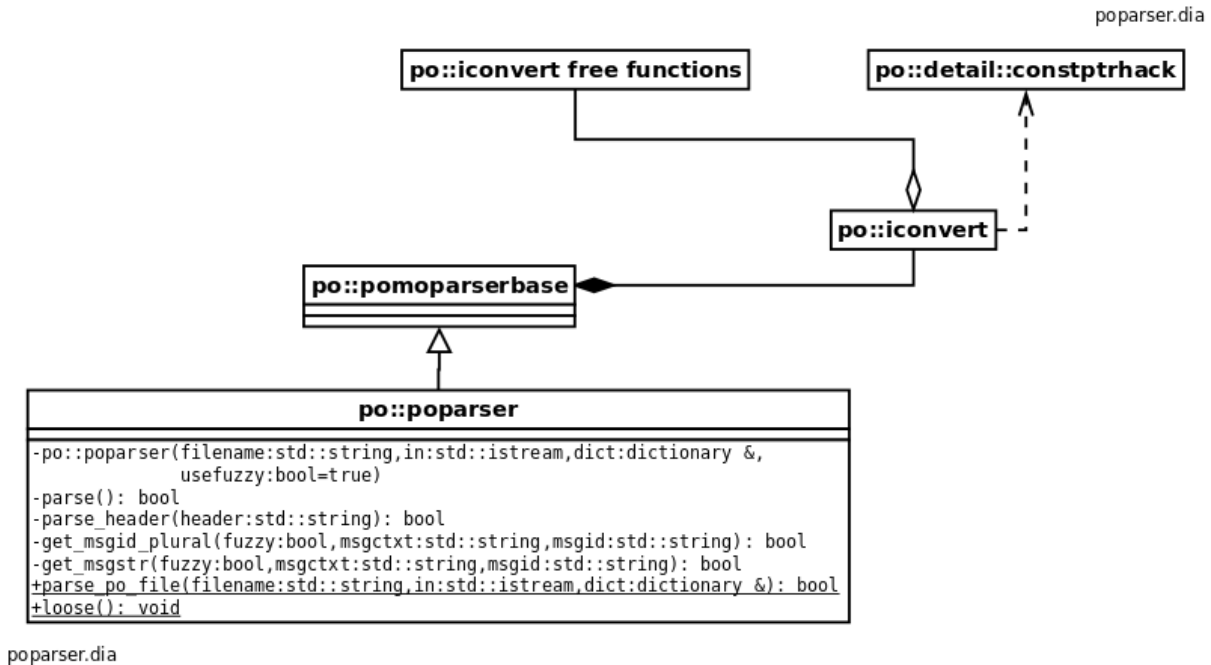


Figure 5: PO Parser (`po::poparser`)

The `poparser` "connects" a `.po` file, an input stream, and a `dictionary` object in order to populate the dictionary with plural forms, set the character-set, and use it (if needed) to convert the translated message string to the character-set. The static function `po::poparser::parse_po_file()` is called to create a temporary `poparser` and use it to read a file and fill in an empty dictionary.

The `po::poparser::get_string_line()` function handles the main task of parsing a line from the `.po` file and deciding what to do with it.

The `po::poparser::get_msgstr` function adds a message (which might be converted to a specified character-set) to the dictionary.

The `po::poparser::get_msgid_plural()` adds a plural form (see section 3.8 "Pluralforms Class" on page 19) or a contextual translation to the dictionary.

The `po::poparser` class uses the `po::iconvert` class to convert the string translation to the desired character-set. The `po::iconvert` class is a reimplementaion of the `tinygettext::IConv` class. Note that it defines the `po::iconv_t` type.

3.5 Moparser Class

The `po::moparser` class is based on `pomoparserbase`, and adds the ability to parse `.mo` files. `tinygettext::POParser` class.

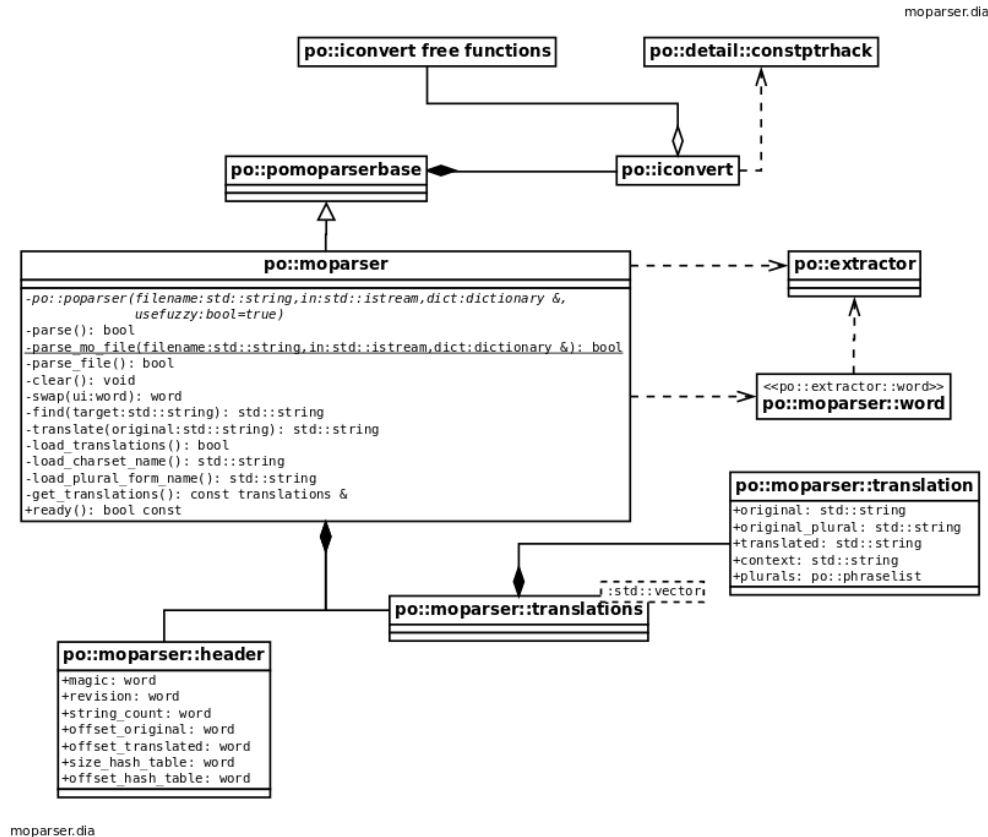


Figure 6: MO Parser (`po::moparser`)

The `moparser` "connects" a `.mo` file, an input stream, and a dictionary object in order to populate the dictionary with plural forms, set the character-set, and use it (if needed) to convert the translated message string to the character-set. The static function `po::moparser::parse_mo_file()` is called to create a temporary `moparser` and use it to read a file and fill in an empty dictionary.

The format of the `.mo` file is described in the `moparser` module's top banner. The format is dissected and described further in these files:

```

library/tests/mo/es/newt.hex
library/tests/mo/de/helloworld.hex

```

Note the `moparser::header` structure, which provides the layout of the first few words in the `.mo` file. Each `translation` structure holds the "original" message ID, the "original plural" message ID, the context (if specified), the translated singular string, and a list of plural forms (if specified).

The `moparser` class gets the character-set name and the plural-forms description by searching for

the key strings to find them. It then searches for original singular strings, original plural strings, context strings, and the list of plural translations. These items are stored in a vector of translations, which is later used to populate a `dictionary`.

The process of the binary data is facilitated by the `extractor` class discussed in the next section.

3.6 Extractor Class

This new class is used internally in the parsing of `.mo` files. It encapsulates some tricky binary data extraction so that the `moparser` class is less complex than otherwise. The binary data is represented by a string, which is loaded by the constructor.

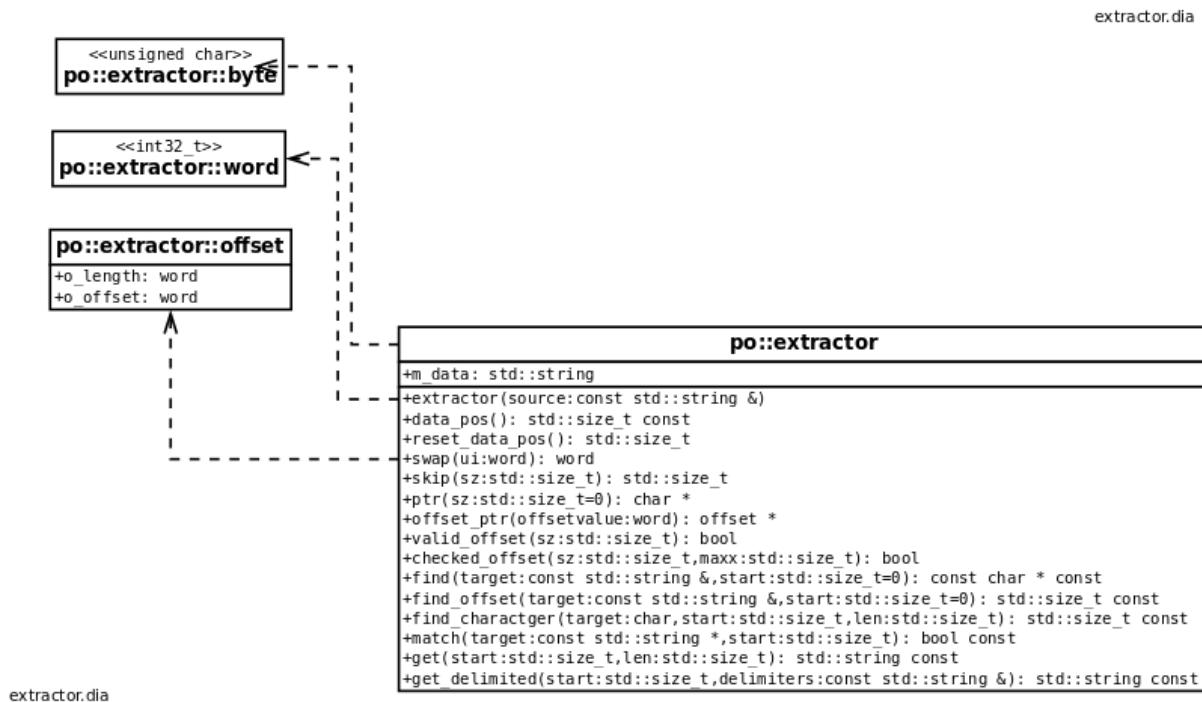


Figure 7: Binary Data Extractor (`po::extractor`)

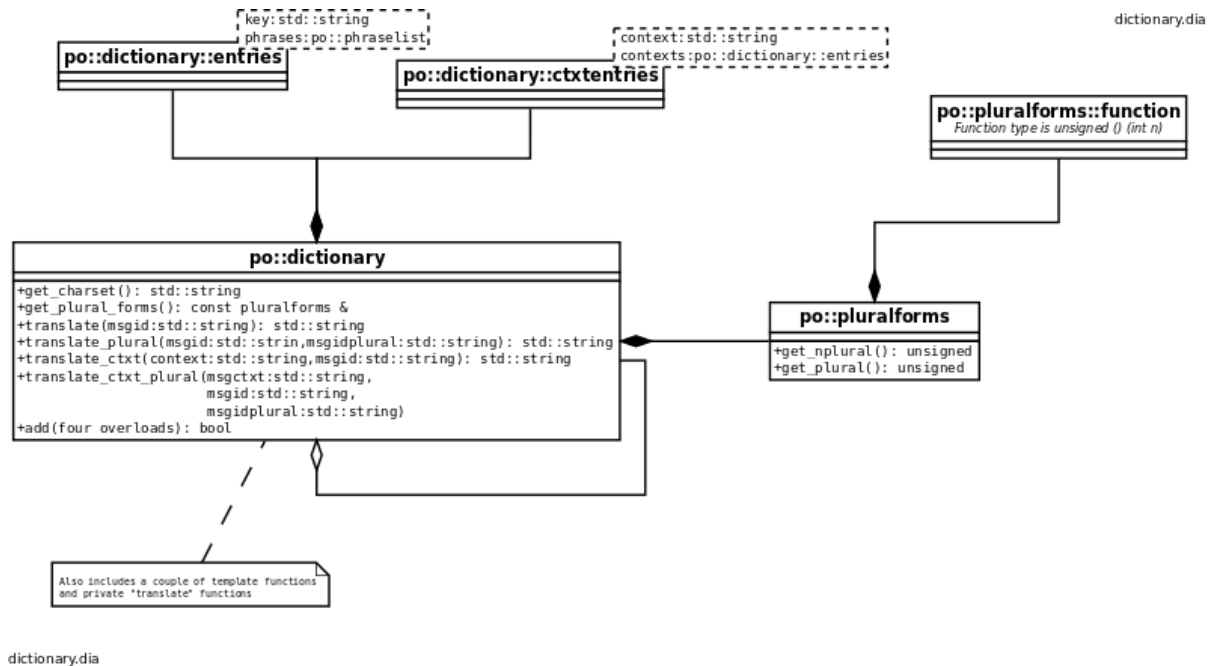
For the most part, the data in the string is accessed by position and length. Offsets and pointers for string or character targets can be found using a starting offset, a length, and perhaps a character delimiter such as a newline, null byte, and the EOT (ASCII 4) character.

Note the data types of `byte` and `word`, here hidden in the extractor class.

Also included is the `offset` structure, which is the pair of data items the `.mo` format uses.

3.7 Dictionary Class

The `po::dictionary` class is a reimplementation of the `tinygettext::Dictionary` class.

Figure 8: Dictionary (`po::dictionary`)

The `dictionary` class holds a set of conversions of strings to a list of possible conversions, and another set of lists to support various message contexts.

The dictionary contains `entries`, an `std::unordered_map` of phrases keyed by a message ID string as used in *GNU gettext*. A `phraselist` is simply an `std::vector<std::string>`.

The dictionary also contains `ctxtentries`, an `std::unordered_map` of `entries` keyed by a context string.

The dictionary also contains a `pluralforms` object that can be used to look up the proper plural translation. These functions provide the desired lookups:

- `translate()`.
- `translate_plural()`.
- `translate_ctxt()`.
- `translate_ctxt_plural()`.

Note that there are no functions that use the name of a domain as a parameter. Instead, the domain-using functions in the `gettext` module look up the dictionary associated with the desired domain, and use the appropriate translate function.

A new function `create_po_dump()` generates a string that is a reasonable representation of a `.po` file. It can be used to convert a `.mo` file to a `.po` file. An example is its usage in the `potext_test list-msgstrs` command.

If one does not need this functionality, undefine this macro in the `po_build_macros.h` header file:

```
POTEXT_DICTIONARY_CREATE_PO_DUMP
```

3.8 Pluralforms Class

The `po::pluralforms` class is a reimplementation of the `tinygettext::PluralForms` class. Each `.po` file contains a line describing how the translation of plurals is to be handled for each language:

```
Plural-Forms: nplurals=2; plural=n != 1;
```

The `pluralforms` class provides a static function for each possible plural form (and there are quite a number of them). These functions are inserted into an `std::unordered_map` which is keyed by strings like the one shown above. Some of these strings are extremely long. (*We could shorten the keys by ignoring the redundant part of the plural-forms string.*)

The `po::pluralforms::from_string()` function strips the spaces from a string parameter and does a fast lookup to return the appropriate `pluralforms` object.

3.9 Language Class

The `po::language` class is a reimplementation of the `tinygettext::Language` class.

The `po::languagespec` structure is a reimplementation of the `tinygettext::LanguageSpec` structure. This structure now uses `std::string` instead of character pointers.

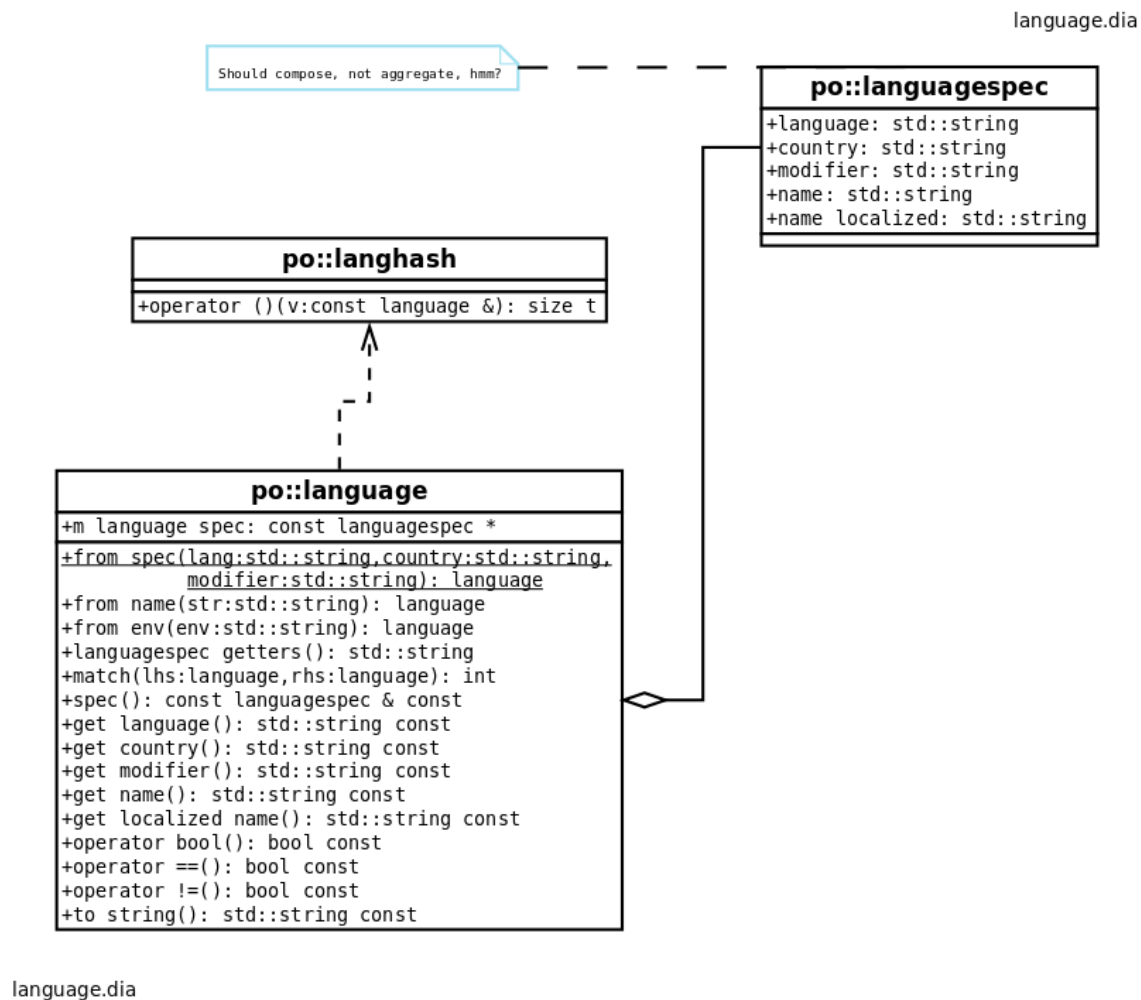


Figure 9: Language (po::language)

The `language` class is a wrapper for a `languagespec` structure. As shown in the figure, it provides functions to get and set the components of a language specification, to make comparisons, and converted the specification to a string.

The `dictionarymgr` uses the `language` as a key to look up the desired dictionary, and if not found, to make a new dictionary and add it to the dictionary container.

We still need to understand a little more about this class and its usage.

3.10 NLS Bindings Class

The `nlbindings` class is an addition for the *Potext* library to support adding text-domain functions akin to those in *GNU Gettext*, but wrapped in the `po` namespace.

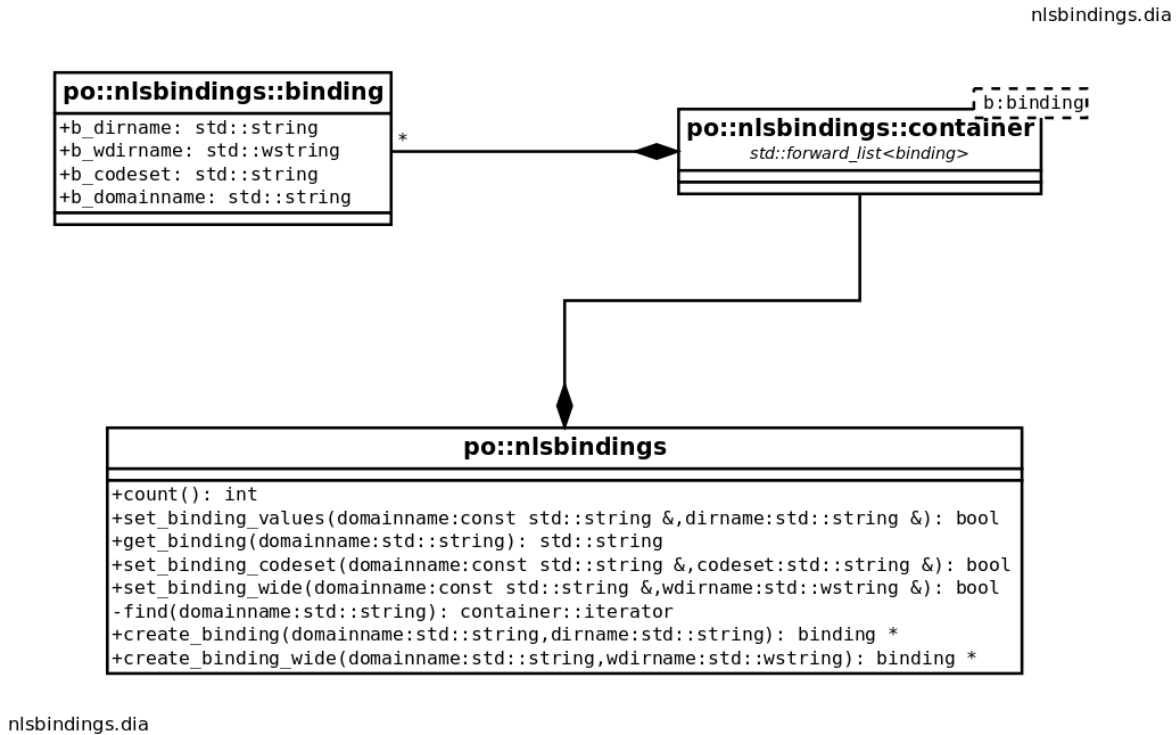


Figure 10: NLS Bindings (po::nlsbindings)

It provides some simplified implementations of *GNU Gettext* functions that lack such niceties as locking and checking for SUID root applications. These can be added later as the need becomes apparent. For details, see the code in the *GNU Gettext* project in its `gettext-runtime/intl` directory.

3.11 Gettext Module

The *Potext* `gettext` module is discussed in the next section. (See section 4.2 "Gettext Module" on page 22.)

4 GNU Gettext and Its Potext Replacements

This section briefly covers the public functions and macros of *GNU Gettext* and our replacements for them. Here are the main differences:

- All implementations are functions; no macros are used.
- All functions are inside the `po` namespace.
- All character pointers are replaced by `std::string`.
- Lookups are done via `.po` files, at present.
- None of the functions with a "category" parameter are implemented at this time. Those function would seem to need to find and load up another `dictionary` object. Also, by far the most common translation files on a *UNIX* system are in the `LC_MESSAGE` subdirectories.

4.1 GNU Gettext Header File

This section provides a walkthrough of the `gettext.h` header file of the *Potext* library. This is useful in understanding *Gettext* versus *Potext*.

Let us survey the important functions and macros that are used in the `gettext.h` header file (see `/usr/include/libintl.h`):

- `ENABLE_NLS`. If defined in a GNU automake project, this includes the `libintl.h` header file, which is not needed in an application using the *Potext* library for translation. NLS can be disabled via `-disable-nls`.
- `DEFAULT_TEXT_DOMAIN`. If `ENABLE_NLS` is defined, this macro causes `gettext` to be defined as `dgettext`, and `ngettext` to be defined as `dngettext`. If `ENABLE_NLS` is *not* defined, then the following "functions" are "voided":
 - `gettext`
 - `dgettext`
 - `dcgettext`
 - `ngettext`
 - `dngettext`
 - `dcngettext`
 - `textdomain`
 - `bindtextdomain`
 - `bind_textdomain_codeset`
- `DEFAULT_TEXT_DOMAIN` revisited. If defined, more macros are defined, for message-context support. These call `pgettext_aux` or `npgettext_aux`
 - `pgettext`
 - `dpgettext`
 - `dcpgettext`
 - `npgettext`
 - `dnpgettext`
 - `dcnpgettext`
- `GNULIB_defined_setlocale`. If defined, uses the `rpl_setlocale` from *gnulib* as `setlocale`.
- `gettext_noop`. A pseudo function that marks code for extraction of messages, but does not call `gettext`.
- `pgettext_expr`. Calls `dcpgettext_expr()`.
- `dpgettext_expr`. Calls `dcnpgettext_expr()`.

Do we want *potext* to be a drop-in replacement for all this stuff? We shall try!

4.2 Gettext Module

The `gettext` module provides a reimplementation of GNU *Gettext* "gettext" functions in the `po` namespace.

Here, we use the term "module" to describe a set of related functions that are not members of

a class. All functions in this module are in the `po` namespace, or are `static` and internal to the module.

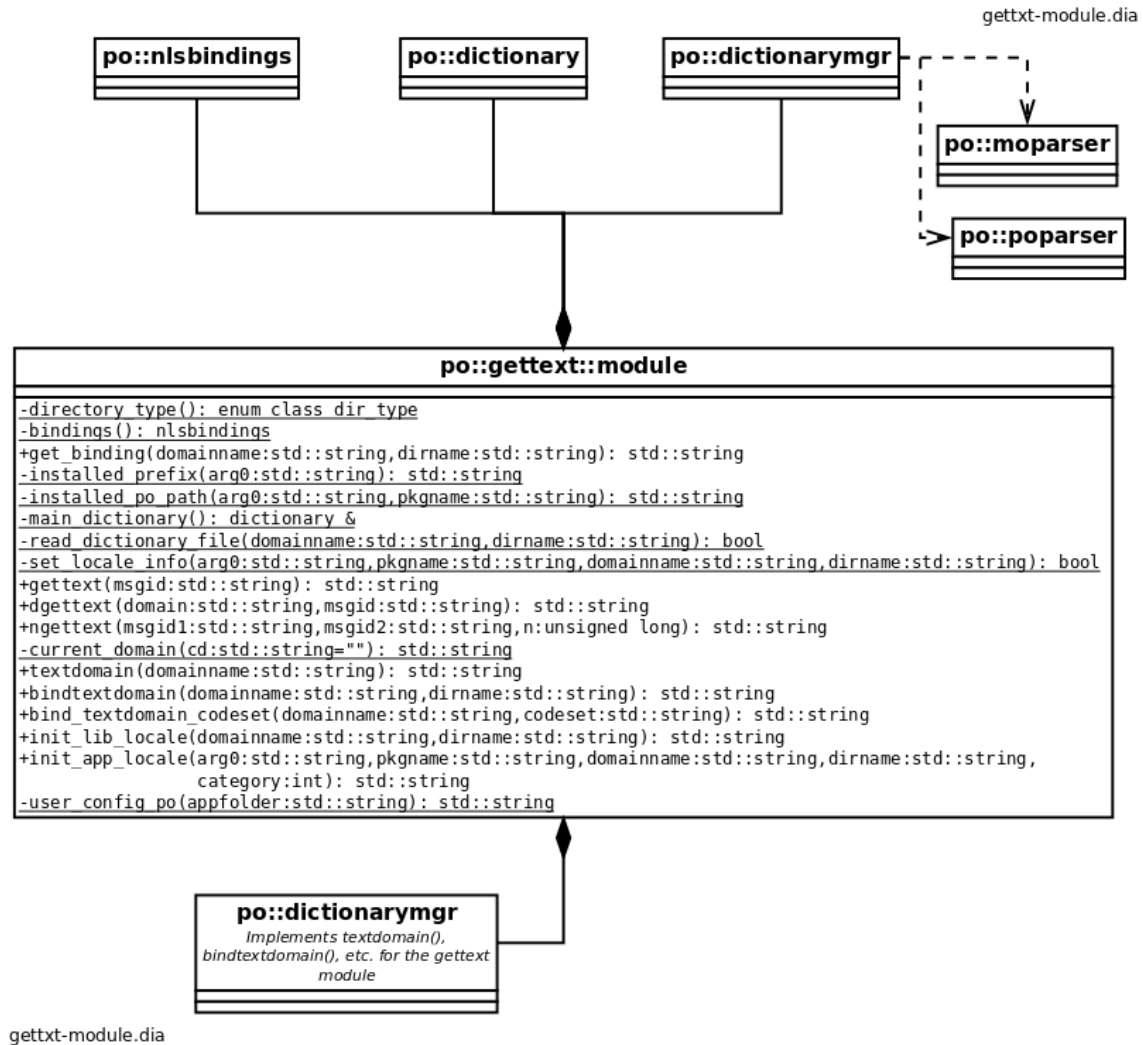


Figure 11: Gettext Module (namespace po)

We are slowly implement the various "gettext" functions shown in that figure, plus some others that are not shown. See the next section for a brief discussion of our copy of the GNU `Gettext` header file.

The following table lists the functions/macros and their purpose and status. The implementations are member functions of `po::dictionary` or `po::dictionarymgr` (*); all dictionaries are contained and referenced in `po::dictionarymgr`, either as the main dictionary or a dictionary selected based on domain. Fall-back functions are noted for some "none" implementations.

Table 1: Gettext Functions

Function	Implementation	Purpose
<code>textdomain()</code>	<code>textdomain()</code> *	Set or change the current global domain for the <code>LC_MESSAGE</code> category.
<code>bindtextdomain()</code>	<code>bindtextdomain()</code> *	Set or change the locale directory for the given domain. <code>LC_MESSAGE</code> category. The wide-character UTF-16 version for Windows is not yet implemented.
<code>bind_textdomain_codeset()</code>	partial *	Set or change the character-set for the given domain. <code>LC_MESSAGE</code> category.
<code>gettext()</code>	<code>translate()</code>	Single message ID translation.
<code>dgettext()</code>	<code>translate()</code>	Single message ID translation in a specific domain.
<code>dcgettext()</code>	none: <code>dgettext()</code>	Single message ID translation in a specific domain and specific language category. For all get-text functions with a category parameter, there is currently no implementation, just a fall-back to the non-category version.
<code>ngettext()</code>	<code>translate_plural()</code>	Message ID translation using a specific singular or plural form.
<code>dngettext()</code>	<code>translate_plural()</code>	Message ID translation using a specific singular or plural form for a given domain.
<code>dcngettext()</code>	none: <code>translate_plural()</code>	Message ID translation using a specific singular or plural form for a given domain and a given locale category.
<code>pgettext()</code>	<code>translate_ctxt()</code>	Single message ID translation for a given context (e.g. "console" versus "gui". The 'p' stands for 'particular'.
<code>dpgettext()</code>	<code>translate_ctxt()</code>	Single message ID translation for a given context and the given domain.
<code>dpcgettext()</code>	none: <code>dpgettext()</code>	Single message ID translation for a given context, given domain, and category other than <code>LC_Messages</code> .
<code>npggettext()</code>	no	Probably worth doing.
<code>dnpggettext()</code>	no	Probably worth doing.
<code>dcnpggettext()</code>	no	Probably not worth doing.

5 Potext Tests

This section provides a useful walkthrough of the testing of the *potext* library. They illustrate the various ways in which the *Potext* library can be used by a developer.

These tests are not installed; they are in the transitory directory `potext/build/library/tests`. The test `*.po` files are in the directory `potext/library/tests` and its subdirectories. The shell script `library/tests/tests.sh` runs all of the `potext_test` tests listed in the following file, plus a couple more.


```
library/tests/testlines.list
```

It specifies the tests described here, but using only single-word phrases. The reason is that we have not yet figured out how to deal with phrases enclosed in double-quotes in a shell script.

Let us survey the main features of all of the test `po` files:

- `po` directory. This directory contains a small sampling of `.po` files from the *GNU Gettext* project. They have been pared down a bit just to save a few bytes, and a few extra translations have been added for testing. They are used in the new `helloworld` test program to test the functions in the `gettext` C++ module.
- `library/tests/de.po`. A basic `po` file with just `msgid` keys and `msgstr` translations in Deutsche (German, Alemania).
- `library/tests/broken.po`. This file has an entry `msgstr[10]`, obviously bad.
- `library/tests/helloworld/de.po`. It has some `msgctxt` sections with `msgid_plural`, `msgstr[0]` for a singular translation and `msgstr[1]` for a plural translation for each context (none, "gui", and "console").
- `library/tests/level/de.po`. Contains basic entries plus a number of entries with a blank message-ID followed by a long description and a message-string with a blank value followed by a long translation. NEED TO FIGURE THAT OUT. Also includes a couple of `printf()` format statements.
- `library/tests/po/de.po`. Another basic file with a number of translations and a weird message-ID called "umlaut".
- `library/tests/po/de_AT.po`. A short file with "umlaut" and a couple of plurals.
- `library/tests/po/fr.po`. Contains one German translation. Wtf?

One thing to watch for in running the tests. The test programs write output to `std::cout` or `std::cerr`, while the *Potext* library internals use the `po::logstream` class functions. What happens is that all of the application output comes first, while the log-stream messages are not emitted until test program exit. Not sure why the latter aren't "flushed" immediately.

Here are the tests provided:

- `helloworld`
- `helloworld`
- `po_parser_test`
- `mo_parser_test`
- `potext_test`

The code is in the `library/tests` directory, and the executables are written to `build/library/tests` directory.

5.1 Hello World

`helloworld`. This test is the original from the *tinygettext* project. Not much done with it; it includes a copy of the `gettext.h` header file and calls some of the stock *GNU Gettext* functions.

5.2 Hello Potext

hellopotext. This application is a good test of *potext* from the perspective of a developer wanting to use it in an application. Without arguments, this test runs through basic tests of the following functions. The main domain is provided to `po::init_app_locale()` which should normally be called in the applications's `main()` entry point function.

- `_()` and `gettext()`. This function does a message translation lookup using the current domain, which is logged in the `init_app_locale()` function. This smoke test illustrates the most common case we want to cover, which is the translation of a phrase according to the main (or only) domain and locale directory loaded. Also included is a test where the original message is not present in the `.po` file.
- `dgettext`. This function looks up a domain's dictionary and uses it for the translation. This test runs through all of the domains (i.e. `.po` files) in the `po` project directory. Currently tested are the *es*, *fr*, *de*, and a bogus domain named *xx*, which should just return the input message ID.
- `dcgettext`. This test does not do anything. Currently *Potext* does not handle locale categories. The reasons? First, the most common use case is looking up message translations in the `LC_MESSAGES` locale category. Second, this translation would require loading additional locale directories and their dictionaries. With this complication, we will sit on this problem for awhile.
- `ngettext`. This test handles plural forms in the current domain. It deals only the the main domain, *es*. It tests translating the plurals of the following singulars: "File" and "Person". The translations are likely not accurate, as they were provided by *Google Translate*. But they adequately test the process.
- `dngettext`. This test handles plural forms in a specified domain. Currently tested are the *es*, *fr*, *de*, and a bogus domain named *xx*, which should just return the input message ID.
- `dcngettext`. This function is not yet implemented, due to difficulties with selecting the category directory, as discussed above.
- `pgettext`. This test applies only to the domain specified in the `init_app_locale()` function.

Some functions not yet tested because of the implmentation difficulties noted above.

- `dcgettext()`.
- `dcngettext()`.

Additional arguments can change the default domain. We will document these *real soon now*.

5.3 PO Parser Test

po_parser_test This small application tests the ability to parse some sample `.mo` files.

To run this test, simply supply the name of a `.po` file and examine the results. Any `.po` file can be provided.

Without arguments, this application lists some options and stock test `.po` files to run.

5.4 MO Parser Test

mo_parser_test This small application tests the ability to parse some sample `.mo` files.

To run this test, simply supply the name of a `.mo` file and examine the results. Any `.mo` file can be provided.

Without arguments, this application lists some options and stock test `.mo` files to run.

In progress. We need to add some more files to be able to test all of the most important aspects of `.mo` files.

5.5 Potext Test

potext_test This application is another good test of *potext* from the perspective of a developer wanting to use it in an application. Running it without any arguments shows a list of 8 tests. These tests are run by supplying the appropriate command-line arguments. These are reflected in the following sections.

5.5.1 Potext Test 'Translate'

These tests are run using a command like the following:

```
$ ./build/library/tests/potext_test translate <...options...>
```

By running **potext_test** without any options, one sees four "translate" commands. The four variations on the "translate" test are described in the following sub-sections.

5.5.1.1 Translate Basic: Po File and Sample Message

This test is a simple translation of a word. The basic "translate" test is run by the following form, which has an argument count of 4.

```
$ ./build/library/tests/potext_test translate <file> <msg>
```

The file is a test `.po` file in the `tests` directory. The message is a phrase present in that file, such as *"F1 - show/hide this help screen"*, translated in `de.po` as *"F1 - Hilfe anzeigen/verstecken"*.

Here is the run:

```
$ ./build/library/tests/potext_test translate \  
  library/tests/de.po "F1 - show/hide this help screen"
```

The output is

```
Translation: "F1 - Hilfe anzeigen/verstecken"
```

If only a part of the message is provided, of course there is no match, and the message is

```
Translation: "F1 - Hilfe"
[potext] Couldn't translate: "F1 - Hilfe"
```

This second test shows that any deviation from a supported message causes an warning, and returns the original message. These deviations include missing letters, missing punctuation, additional spaces. *We wonder if we can work around such issues in this library. We shall see.*

5.5.1.2 Translate: Po File, Context, and Sample Message

This test is run by the following form, which has an argument count of 5.

```
$ ./build/library/tests/potext_test translate <file> <context> <msg>
```

This test requires a po file with message-context entries such as these three different entries found in `library/test/helloworld/de.po` for the phrase "Hello World":

```
msgctxt ""
msgid "Hello World"
msgctxt "console"
msgid "Hello World"
msgctxt "gui"
msgid "Hello World"
```

Please note that the *GNU gettext()* documentation says that an empty message context (`msgctxt ""`) is *not* the same as a missing message context. In the "helloworld" test program, these contexts are provided by the following lines:

```
pgettext("", "Hello World")
pgettext("console", "Hello World")
pgettext("gui", "Hello World")
```

The macros `ngettext` and `npgettext` are also used to provide access to the various plural forms in that po file. In any case, we need to use `library/test/helloworld/de.po` for this test. An actual test run:

```
$ ./build/library/tests/potext_test translate \
    library/tests/helloworld/de.po "console" "Hello World"
```

The output is

```
Context 'console' translation: "Hallo Welt (singular) in der Console"
```

If the <context> parameter is not found in the po file, a message is emitted to indicate the error.

5.5.1.3 Translate: Po File with Singular and Plurals

This test is run by the following form, which has an argument count of 6.

```
$ ./build/library/tests/potext_test translate <file> <singular>
    <plural> <N>
```

The singular and plural parameters are message IDs, such as "Hello World". This command is a bit tricky; the N value is not a C index, but an index that starts at 1. The N parameter ranges from 1 to the last array value in the po file. The number of singular/plural translations depends on the language and is specified in the specific .po file using a header declaration such as "Plural-Forms: nplurals=2; plural=(n != 1);". Look at pluralforms.cpp to see all the plural-forms settings and "callbacks" that are support. Some of these forms support Slavic and Arabic languages, and we are not able to test them.

An actual test run:

```
$ ./build/library/tests/potext_test translate \
    library/tests/helloworld/de.po "Hello World" "Hello Worlds" 1
```

The output is

```
TODO
```

5.5.1.4 Translate: Po File with Context, Singular, and Plurals

This test is run by the following form, which has an argument count of 7.

```
$ ./build/library/tests/potext_test translate <file> <context> \
    <singular> <plural> <N>
```

An actual test run:

```
$ ./build/library/tests/potext_test translate \
    library/tests/helloworld/de.po "gui" "Hello World" "Hello Worlds" 0
```

The output is

```
TODO
```

5.5.2 Potext Test 'Directory'

These tests are run using a command like the following:

```
$ ./build/library/tests/potext_test directory <dir> <msg> [<lang>]
```

5.5.3 Potext Test 'Language'

These tests are run using a command like the following:

```
$ ./build/library/tests/potext_test language <lang>
```

5.5.4 Potext Test 'Language Directory'

These tests are run using a command like the following:

```
$ ./build/library/tests/potext_test language-dir <dir>
```

5.5.5 Potext Test 'List Message Strings'

These tests are run using a command like the following:

```
$ ./build/library/tests/potext_test list-msgstrs <file>
```

The "file" parameter can be a .po file or a .mo file. This test takes the dictionary read in from that file and basically writes a .po file to the console.

6 Summary

Contact: If you have ideas about *Potext* or a bug report, please email us (at <mailto:ahlstromcj@gmail.com>).

Remaining issues:

The *.po files "msgid" and "msgstr" entries have punctuation marks and trailing spaces that are significant. CAN WE GET AROUND THIS ISSUE? We need to trim these trailing characters in both specifications, and also when translating, and restore them in the translation.

7 References

The *Potext* reference list.

References

- [1] GNU Translation Team. *GNU Gettext Code* <https://git.savannah.gnu.org/git/gettext.git>. 2023.
- [2] GNU Translation Team. *GNU Gettext Tools manual, version 0.22*. <https://www.gnu.org/software/gettext/manual/gettext.pdf>. 2023.
- [3] Poedit Team. *Gettext translation for PHP, Python, etc.* <https://poedit.net/>. 2023.
- [4] Polib Team. *Gettext file creation, manipulation, etc. for Python* <https://pypi.org/project/polib/>. 2023.
- [5] Laurent Cozic *Simple Gettext on GitHub*. <https://github.com/laurent22/simple-gettext>. 2017.
- [6] Tinygettext Team. *Tinygettext on GitHub*. <https://github.com/tinygettext/tinygettext>. 2023.